PRACTICAL ASSIGNMENT

Secure Student Management REST API

Topic: Authentication & Authorization
Stack: Java + Spring Boot, Spring Security, PostgreSQL, JWT, Maven, Postman

> **Objective:** Build a secure REST API that authenticates users, protects endpoints, and authorizes actions according to user roles.

1. General Requirements

Build the application using Spring Boot and PostgreSQL. Use a clean layered architecture:

```
Controller
↓
Service
↓
Repository
↓
PostgreSQL
```

Use Git for version control and provide a README explaining how to run and test the application.

2. User Registration

Implement:

```
POST /api/auth/register
```

Registration must include username, email, and password. Username/email should be unique as appropriate. Passwords must be securely hashed before storage. Plain-text passwords are not allowed.

3. Login

Implement:

```
POST /api/auth/login
```

Successful login must return an access token and a refresh token. Invalid credentials must be rejected.

4. JWT Authentication

Protect the Student API using JWT. Clients must send the access token as a Bearer token:

```
Authorization: Bearer <access-token>
```

Protected endpoints must reject requests without valid authentication.

5. Required Roles

Create these roles:

```
ADMIN
TEACHER
STUDENT
```

6. Authorization Rules

Operation	ADMIN	TEACHER	STUDENT
View students	YES	YES	YES
Create student	YES	YES	NO

Update student	YES	YES	NO
Delete student	YES	NO	NO
View/manage users	YES	NO	NO

> Authentication and authorization are different: an authenticated user may still be forbidden from performing an operation.

7. Student CRUD

Implement the following endpoints:

```
POST /api/students
GET /api/students
GET /api/students/{id}
PUT /api/students/{id}
DELETE /api/students/{id}
```

A student should contain appropriate fields such as id, registrationNumber, fullName, email, phone, gender, and dateOfBirth. You may add other useful fields.

8. Refresh Token

Implement:

```
POST /api/auth/refresh
```

A valid refresh token must be able to obtain a new access token. Do not use the refresh token as the normal credential for ordinary protected API requests.

9. Logout

Implement:

```
POST /api/auth/logout
```

Provide a reasonable strategy for invalidating or revoking refresh-token usage after logout and explain it in the README.

10. Validation & Exceptions

Validate required fields, email format, and password requirements. Implement centralized exception handling for user/student not found, duplicate data, invalid credentials, validation errors, unauthorized requests, and forbidden operations.

11. CORS & CSRF

Configure CORS appropriately for a frontend on another origin and document the allowed development origin. Make an intentional CSRF configuration based on your authentication design and explain the decision. Do not blindly disable security features.

12. Database & ERD

Use PostgreSQL. At minimum, create entities/tables for users and students, with roles represented appropriately. Submit an ERD showing the main relationships.

13. Suggested API Structure

```
/api/auth/register
/api/auth/login
/api/auth/refresh
/api/auth/logout
```

```
/api/students
/api/students/{id}

/api/users
```

You may use a different RESTful structure if it is clearly documented.

14. Postman Testing

Create a Postman collection and demonstrate:

■ Register a user
■ Attempt duplicate registration
■ Successful login
■ Incorrect login
■ Protected endpoint without token
■ Protected endpoint with valid token
■ ADMIN endpoint as STUDENT
■ Create student as TEACHER
■ Attempt delete as TEACHER
■ Delete as ADMIN
■ Refresh access token
■ Logout/revocation behavior
■ Invalid request data
■ Student not found
■ CORS behavior

15. Deliverables

Submit:
1. Complete Spring Boot source code.
2. PostgreSQL setup/migration scripts.
3. ERD.
4. Postman collection.
5. README.md.
6. Screenshots/exported results of important tests.
7. Short explanation of authentication, authorization, JWT, roles, refresh tokens, CORS, and CSRF.

16. README Requirements

Include project description, technologies, prerequisites, database configuration, setup instructions, endpoint list, authentication instructions, roles/permissions, Postman instructions, refresh-token strategy, CORS configuration, and CSRF configuration.

17. Assessment Rubric

Area	Marks
Registration + password hashing	10
Login + authentication	10
JWT access token	15
Refresh token + logout/revocation	10
Roles + authorization	15
Student CRUD	10
Validation + exception handling	10
CORS + CSRF	5
Postman testing	5
ERD + README + code organization	10
Total	**100**

18. Demonstration Questions

During the demonstration, students should be able to explain:

1. Authentication vs authorization.
2. Why passwords are hashed.
3. What JWT is.
4. Access vs refresh tokens.
5. What happens inside Spring Security when a request arrives.
6. Why a user receives 401 or 403.
7. How roles control access.
8. What CORS solves.
9. What CSRF solves and why it matters for the chosen authentication mechanism.
10. How logout affects refresh-token usage.

19. Success Criteria

The application must run successfully from the README instructions. Students must demonstrate the security behavior in Postman, not merely submit code. They should be able to trace a request from the client through authentication and authorization to the controller/service/repository layers.

Core outcome: Authentication establishes identity; authorization controls access. The student should be able to explain why every protected request is allowed or rejected.